Inhaltsübersicht

1. Einleitung und thematische Annäherung
2. Grundlagen, Grundkonzepte und Architektur von Datenbanksystemen
3. Konzeptioneller (fachlicher) Datenbank-Entwurf und das Entity-Relationship-Modell
4. Das relationale Datenmodell
5. SQL
   1. SQL im Überblick
   2. Zentrale SQL DDL-Anweisungen
   3. Zentrale SQL DML-Anweisungen
   4. Zentrale SQL DCL-Anweisungen
6. Relationaler Schemaentwurf
7. Datenbank-Transaktionen und Transaktionsverwaltung
8. Physische Datenbank-Organisation
9. Ausblick auf weiterführende DBS-bezogene Themen

SQL – Historie & Allgemeine Information

- **SQL: Structured Query Language**

- **Entstanden als Datenbanksprache für einen RDMBS-Prototyp („System R“, 1976) von IBM**
  - **Beginn der Entwicklung Anfang der 70er Jahre**
  - **Ursprüngliche Bezeichnung: SEQUEL (Structured English QUEry Language)**

- **Erste kommerziell verfügbare Systeme:**
  - **1979 – erstes SQL-basiertes DBMS „Oracle“ von Relational Software (später Oracle Corporation)**
  - **1981 – erstes SQL-basiertes DBMS von IBM: SQL/Data Storage (SQL/DS)**
  - **1983 – weiteres RDBMS von IBM: DB2**
  - **1985 – Informix und Ingres unterstützen mit ihren DBMS auch SQL**

- **Heutzutage ist SQL <u>die</u> Datenbanksprache für relationale DBS, „quasi-monopolartige“ Verbreitung**

SQL – Historie & Allgemeine Information

- **Wesentlicher Grund für Verbreitung: *Standardisierung***
  (gemeinsam durch ANSI und ISO bzw. ISO/IEC)

  - **1986 Erste Standardversion: SQL-86 (auch als „SQL1“ bezeichnet, ANSI-Norm)**
  - **1992 deutliche Erweiterung (und Modifikationen) des Standards: SQL-92**
    (auch bezeichnet als „SQL2“, erste gemeinsame ISO / ANSI-Norm; nachfolgende Versionen als internationale ISO/IEC-Normen)
  - **1999 Erweiterung des Standards u.a. um objekt-relationale bzw. objektorientierte DB-Konzepte: SQL:1999 („SQL3“)**
  - **Unterstützung von XML: SQL:2003 & SQL:2006**
  - **Bisher letzte Versionen: SQL:2008 & SQL:2011 (veröffentlicht im Dezember 2011)**

- **In (kommerziellen) RDBMS am häufigsten verwendet werden derzeit SQL:1999 (und SQL:2003), jedoch meist nicht vollständig implementiert**

SQL – Historie & Allgemeine Information

Kritikpunkte am SQL-Standard:

- **Relativ später Beginn der Standardisierung, d.h. Systeme existierten bereits, als Standard definiert wurde („Standard kam zu spät“)**
- **Teilweise (SQL1 → SQL2) wurden Konzepte wieder entfernt, d.h. einige der Standardversionen sind nicht aufwärtskompatibel; entfernte Konzepte wurden aber in einigen DBS beibehalten**

Praktisch kein RDBMS hat den SQL-Standard in Reinform und im vollen Umfang realisiert.

Jedes kommerzielle RDBMS bietet einen „SQL-Dialekt“ an, mit einem mehr oder minder großen gemeinsamen Sprachkern über die verschiedenen RDBMS hinweg.

→ Im Wesentlichen Unterstützung von SQL-92 und Abdeckung von Teilen der nachfolgenden Standardversionen SQL:1999 und SQL:2003 im jeweiligen „Dialekt“ des RDBMS

SQL – Historie & Allgemeine Information

Drei wesentliche Teilsprachen im SQL-Standard:

- **DDL (Data Definition Language)** → Definition des Schemas (der DB-Struktur)
  - Definition des Schemas
  - Definition von Tabellenstrukturen (Relationstypen) mit Integritätsbedingungen (Schlüssel-, Nullwertbedingung, …)
  - Definition von Domänen (Wertebereiche für Attribute)
  - Definition von Sichten
  - (Definition von Indexstrukturen
    Achtung: Teil des internen, nicht des konzeptionellen Schemas)

- **DML (Data Manipulation Language)** → Lese- und Änderungsoperationen auf Daten
  - Einfügen, Ändern, Löschen von Tupeln (Zeilen)
  - Abfrage von Daten in der Datenbank
  - Definition und Steuerung von Transaktionen

- **DCL (Data Control Language)** → Definition von Zugriffsrechten („Privilegien“)
  - Benutzerverwaltung und Vergabe von Zugriffsrechten

SQL – Historie & Allgemeine Information

Nutzungsmöglichkeiten:

- SQL kann ***interaktiv*** zur Kommunikation mit einem RDBS genutzt werden (Eingabe von SQL-Anweisungen am System)
- Einbettung von SQL-Anweisungen ***in Programmcode*** („embedded SQL“)

Begriffe im relationalen Modell und in SQL:

- Relation → Tabelle (Table)
- Tupel → Zeile (Row)
- Attribut → Spalte (Column)

Achtung:

- Dies bedeutet nicht, dass eine SQL-Tabelle (→ prinzipiell doppelte Zeilen möglich) dasselbe ist wie eine Relation (→ keine doppelten Tupel möglich)
- Die Begriffe im SQL-Standard (und in den Büchern bzw. Dokumentationen zu relationalen DBS) sind oft unsauber verwendet. Beispiele:
  - Es wird von Datenbank gesprochen, obwohl die DB-Struktur (=Schema) gemeint ist
  - Es wird von Spalte gesprochen, obwohl nur das Attribut (die „Spaltenüberschrift“) und nicht der Inhalt (=Daten) einer Spalte gemeint ist

SQL: Wesentliche Eigenschaften und Unterschiede zu Programmiersprachen

- **Klassische imperative bzw. prozedurale Programmiersprachen:**
  - **Verarbeitung pro Datensatz**
  - **Bearbeitung einer Menge von Daten(sätzen) erfordert Schleife im Programmcode**

    Beispiel:

```
Öffne Datei
Wiederhole solange Dateiende nicht erreicht
     Lese nächsten Datensatz
     Bearbeite Datensatz
Ende Wiederholung
```

- **SQL:**
  - **Deklarative, nicht-prozedurale Sprache**
    - **SQL-Anweisungen geben an, <u>was</u> als Ergebnis gewünscht („deklarativ“) ist und <u>nicht</u>, <u>wie</u> (=prozedural) das Ergebnis berechnet werden soll**
    - **Gewünschte Daten werden durch logische Bedingungen (Prädikate) beschrieben.**
    - **Beispiel: „Zeige alle Kunden die zwischen 15.04.2015 und 14.07.2015 Waren für insgesamt mindestens 2500 Euro bestellt haben.“**
  - **Keine (Einzel-)satzweise, sondern mengenorientierte Verarbeitung**
    - **SQL verarbeitet grundsätzlich <u>Mengen</u> von Zeilen**
    - **Das Ergebnis einer SQL-Anfrage ist immer eine Menge (die auch leer sein oder aus nur einer Zeile bestehen kann)**

➡ ***„Bruchstelle“ bzw. Lücke zwischen Programmiersprachen und SQL!***

Hinweise zum Inhalt dieses Kapitels

Im Folgenden werden die grundlegenden SQL-Anweisungen betrachtet:

- **… möglichst unabhängig von einer SQL-Version**
- **… beschränkt auf die wichtigsten Optionen pro Anweisung**
- **… möglichst systemunabhängig (systemabhängige Angaben erfolgen für MySQL)**

Für die detaillierte Darstellung aller SQL-Anweisungen mit allen möglichen Optionen pro Anweisung ist der SQL-Standard heranzuziehen bzw. die Dokumentation des jeweiligen RDBMS. **!**

Beispiel für den vollständigen Umfang einer SQL-Anweisung:
(Syntax für Anweisung CREATE TABLE in MySQL 5.1)

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
    (create_definition,...)
    [table_options]
    [partition_options]
```

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
    [(create_definition,...)]
    [table_options]
    [partition_options]
    select_statement
```

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
    { LIKE old_tbl_name | (LIKE old_tbl_name) }
```

3 Varianten der Anweisung …

… + Definition von create_definition, table_options, …

Legende für die Darstellung der SQL-Syntax

Symbol / Zeichenkette	Bedeutung	Beispiel
`XYZ`	Reserviertes SQL-Schlüsselwort oder -Anweisung	`SELECT, CREATE`
<X>	Platzhalter („Variable“)	<Tabellenname>, <Attributname>
XYZ, Xyz	Eingabe, wählbare Angabe (z.B. Tabellenname, Attribut, …)	*Kunde, Name, KdNr*
<X> | <Y> … | …	exklusive Alternative bzw. Auswahlmöglichkeit (entweder <X> oder <Y>, entsprechend auch für Schlüsselworte etc.)	`NULL` | `NOT NULL` , <Tabellenname> |<Attributname>
{ …}	Zusammenfassung	{ `SCHEMA` | `DATABASE` } , {<Tabellenname> |<Attributname>}
[…]	Optionale Angabe (… kann, muss aber nicht angegeben werden)	[<Tabellenname>] , [`NULL` | `NOT NULL`]
*	Beliebig häufige Wiederholung (keinmal, einmal oder mehrfach)	<Tabellenname> * {<Tabellenname> |<Attributname>}

Hinweis: Bei SQL-Anweisungen wird <u>nicht</u> zwischen Groß- und Kleinschreibung unterschieden. Zur besseren Lesbarkeit und Übersichtlichkeit wird im Folgenden jedoch immer Großschreibung verwendet

SQL: Grundlegende DDL- und DML-Sprachelemente im Überblick

- **SQL DDL: Definition von (Daten-)Strukturen**
  - Definition von Schemata → `CREATE SCHEMA`
  - Definition von Tabellen (…typen!) und Integritätsbedingungen → `CREATE TABLE`
  - Definition von (benutzerdefinierten) Wertebereichen → `CREATE DOMAIN`
  - Definition von Sichten → `CREATE VIEW`
  - Ändern bzw. Löschen von Tabellen, Wertebereichen, Sichten → `ALTER …` bzw. `DROP …`
  - (Definition von Indexstrukturen → `CREATE INDEX`)

- **SQL DML: Datenänderung und -abfrage**
  - Einfügen von Zeilen (Datensätzen) → `INSERT`
  - Ändern von Zeilen (Datensätzen) → `UPDATE`
  - Löschen von Zeilen (Datensätzen) → `DELETE`
  - Abfrage von Daten → `SELECT`
  - Transaktionssteuerung → `COMMIT`, `ROLLBACK`

SQL DDL: CREATE SCHEMA

Erstellen eines Datenbankschemas:

```
CREATE {SCHEMA | DATABASE} <Schemaname>
[[DEFAULT] CHARACTER SET <Zeichensatzname> ]
[[DEFAULT] COLLATE <Sortierung> ];
```

Beispiel:

```
CREATE SCHEMA Vertrieb;
```

Hinweis:

- **Es sind die Schlüsselworte „ `SCHEMA`“ und „`DATABASE`“ zulässig.**
- **Die Verwendung von `SCHEMA` und `DATABASE` ist in Datenbanksystemen uneinheitlich. Es ist jedoch immer die Anlage der Schema-Struktur gemeint, <u>nicht</u> die Anlage von Inhalten (Daten) in der Datenbank**

SQL DDL: DROP SCHEMA

Grundprinzip in SQL:

Ein mit `CREATE` …erstelltes DB-(Struktur-)Objekt kann mit `ALTER` … geändert und mit `DROP` … gelöscht werden.

Konsequenz:
Zu einer `CREATE-Anweisung` existiert somit eine korrespondierende `DROP-` bzw. `ALTER-Anweisung`.

Löschen eines Datenbankschemas:

```
DROP {SCHEMA|DATABASE} <Schemaname> ;
```

Beispiel:

```
DROP SCHEMA Vertrieb;
```

Hinweis:

Mit dem Löschen des Schemas wird auch die gesamte Datenbank, also der gesamte Datenbestand gelöscht!

SQL DDL: Die wichtigsten (vordefinierten) Datentypen

- SQL und relationale DBMS bieten eine Vielzahl von vordefinierten Standard- oder Grunddatentypen
  - Alphanumerische Datentypen
  - Numerische Datentypen
  - Zeit-Datentypen
  - Binäre Datentypen
  - Boolescher Datentyp

- Bezeichnung (und manchmal auch Bedeutung) der Datentypen können von DBMS zu DBMS herstellerspezifisch abweichen
  - Beispiel: Bezeichnung in MySQL für `CHARACTER VARYING`: `VARCHAR`

SQL DDL: Die wichtigsten (vordefinierten) Datentypen

- **Alphanumerische Datentypen**
  - `CHARACTER(<n>)` → Zeichenkette mit <u>fester</u> Länge <n>
  - `CHARACTER VARYING(<n>)` → Zeichenkette mit <u>maximaler</u> Länge <n>; es werden nur die verwendeten Zeichen belegt

- **Numerische Datentypen**
  - `NUMERIC(<m>,<n>)` → Dezimalzahl mit fester Länge <m> und <n> Nachkommastellen
  - `DECIMAL(<m>,<n>)` → Dezimalzahl mit maximaler Länge <m> und <n> Nachkommastellen
  - `INTEGER` → Ganze Zahl (4 Bytes)
  - `SMALLINT` → Ganze Zahl (2 Bytes)
  - `REAL` → Gleitkommazahl, einfache Genauigkeit

SQL DDL: Die wichtigsten (vordefinierten) Datentypen

- **Zeit-Datentypen**
  - `DATE` → Datumsangabe, meist „JJJJ-MM-TT“
  - `TIME` → Zeitangabe, meist „hh:mm:ss“
  - `TIMESTAMP` → Kombination von `TIME & DATE`

- **Weitere Datentypen**
  - `BLOB` (ab SQL:1999) → Binary Large Object, für binäre Daten wie Graphiken
  - `CLOB` (ab SQL:1999) → Character Large Object, unstrukturierter Text
  - `BOOLEAN` → Wahrheitswert

SQL DDL: Frei definierte Wertebereiche (bzw. Datentypen)

Erstellen eines frei definierten („benutzerdefinierten“) Wertebereichs (Domäne):

```
CREATE DOMAIN <Wertebereichsname>
AS <Datentyp> CHECK <Bedingung>;
```

<Bedingung>: Legt Einschränkung des Wertebereichs fest, basierend auf dem Wertebereich von <Datentyp>. Im Prinzip „beliebige“ Ausdrücke mit `VALUE` möglich

Beispiele:

```
CREATE DOMAIN KundenNr
AS INTEGER CHECK (VALUE > 1000);

CREATE DOMAIN Kundenstatus
AS CHAR(1) CHECK (VALUE IN ('W','S','G'));
```

W: Wichtiger Kunde
S: Stammkunde
G: Gelegenheitskunde

Hinweis:

Domänen können wie vordefinierte Datentypen bei der Definition von Tabellen(typen) verwendet werden.

SQL DDL: CREATE TABLE

Erstellen einer Tabelle(nstruktur) mit (einfachen) Integritätsbedingungen:

```
CREATE TABLE <Tabellenname>
  ( <Spaltendefinition> { ,<Spaltendefinition>}* ) ;

<Spaltendefinition> :=   <Spaltenname> <Datentyp>
                         [DEFAULT <Vorgabewert>]
                         [NULL | NOT NULL]
                         [UNIQUE | PRIMARY KEY]
                         [REFERENCES <Tabellenname> [<Spaltenname> ] ]
<Datentyp> kann einer der zuvor aufgeführten Datentypen sein
```

Beispiel:

```
CREATE TABLE Artikel (
    ArtNr        INT  PRIMARY KEY ,
    Bezeichnung  VARCHAR(30) NOT NULL ,
    MengEinh     VARCHAR(10) ) ;
```

SQL DDL: CREATE TABLE

- Die vorgestellte CREATE TABLE Anweisung erlaubt die Angabe einfacher (d.h. aus einer einzelnen Spalte bestehenden) Primär- und Fremdschlüsseln
- Schlüssel aus mehreren Spalten müssen auf Ebene der Tabellendefinition angegeben werden

Erweiterte Syntax hierfür:

```
CREATE TABLE <Tabellenname>
  ( <Spaltendefinition> { ,<Spaltendefinition>}* { ,<Integritätsbedingung>}* ) ;

<Integritätsbedingung> := <Primärschlüsselbedingung>
                        | <Fremdschlüsselbedingung>
                        | <Sonstige Bedingung>

<Primärschlüsselbedingung> := [CONSTRAINT <Bedingungsname>]
                              PRIMARY KEY (<Spaltenname> {, <Spaltenname>}* )
```

SQL DDL: CREATE TABLE

```
<Fremdschlüsselbedingung > := [CONSTRAINT <Bedingungsname>]
                              FOREIGN KEY ( <Spaltenname> {, <Spaltenname>}* )
                              REFERENCES <Tabellenname>
                                          ( <Spaltenname> {, <Spaltenname>}* )
                              [ON UPDATE [NO ACTION|CASCADE
                                          |SET DEFAULT|SET NULL ]]
                              [ON DELETE [NO ACTION|CASCADE
                                          |SET DEFAULT|SET NULL ]]
<Sonstige Bedingung >      := CHECK <Bedingung>
```

Erläuterung der Optionen:

- `NO ACTION` (alternativ: keine Angabe): Änderung des Primärschlüsselwerts wird abgewiesen
- `CASCADE`: Auch die Zeile mit dem Fremdschlüsselwert wird geändert/gelöscht
- `SET NULL`: Fremdschlüsselwert wird auf NULL geändert (sofern erlaubt)
- `SET DEFAULT`: Fremdschlüsselwert wird auf einen (zuvor definierten) Vorgabewert geändert

SQL DDL: CREATE TABLE

Beispiel:

```
CREATE TABLE Bestellung (
        BestNr              INT,
        KdID                VARCHAR(50) NOT NULL,
        BestDatum           DATE NOT NULL,
        LetzteAenderung     TIMESTAMP DEFAULT  current_timestamp,
        CONSTRAINT PkBest PRIMARY KEY (BestNr),
        CONSTRAINT FkBestKunde FOREIGN KEY (KdID) REFERENCES Kunde(KdID)
        ON UPDATE CASCADE   ON DELETE CASCADE);
```

SQL DDL: CREATE TABLE

Beispiel:

```
CREATE TABLE Bestellposition (
        BestNr              INT,
        PosNr               INT,
        ArtNr               INT  NOT NULL,
        Anzahl              INT DEFAULT 1   NOT NULL,
        LetzteAenderung     TIMESTAMP DEFAULT current_timestamp,
        CONSTRAINT PKBestPos PRIMARY KEY (BestNr,PosNr),
        CONSTRAINT FKBestPos_Best FOREIGN KEY (BestNr)
                REFERENCES Bestellung (BestNr)
        ON UPDATE CASCADE   ON DELETE CASCADE,
        CHECK (Anzahl >= 0)   );
```

SQL DDL: ALTER & DROP TABLE

Ändern bzw. Löschen einer Tabelle(nstruktur):

```
ALTER TABLE <Tabellenname>
   <Änderungsdefinition> { ,<Änderungsdefinition>}* ;

 <Änderungsdefinition> := ADD [COLUMN] <Spaltendefinition>
                        | DROP [COLUMN] <Spaltenname>
                        | ADD <Integritätsbedingung>
                        | DROP { PRIMARY KEY | FOREIGN KEY } [<Fremdschlüsselname>]
```

```
DROP TABLE <Tabellenname> [RESTRICT | CASCADE ];
```

Hinweis:

- **DROP TABLE löscht die Tabelle mit allen enthaltenen Daten und zugehörigen Integritätsbedingungen !!! ***
- **Wird auf die zu löschende Tabelle mit Fremdschlüsselbedingungen verwiesen, so sind diese zuvor mit ALTER TABLE aus den referenzierenden Tabellen zu entfernen (alternativ besteht die Option CASCADE für die DROP TABLE Anweisung, die die entsprechenden Fremdschlüsselbedingungen automatisch entfernt).**

SQL DDL: ALTER & DROP TABLE

Beispiele:

```
ALTER TABLE Bestellung
        ADD Status        CHAR(10),
        ADD Bestellsumme  DECIMAL (8,2);
```

```
DROP TABLE  Bestellposition;
```

```
DROP TABLE  Bestellung CASCADE;
```

SQL DDL: CREATE VIEW

- **Mit `CREATE TABLE` erstellte Tabellen(strukturen) beinhalten die gespeicherten Daten der DB**

- **Neben solchen Tabellen können auch Sichten definiert werden, die unterschiedliche „Sichtweisen“ auf die Daten in den Tabellen ermöglichen.**

Idee / Prinzip einer Sicht:

„There‘s how the data is stored, and if the user wants to see it differently, then you ought to present him with what‘s called a virtual relation – a table that doesn‘t really exist, but can be derived from the actual stored data.“

Quelle: Interview mit Michael Stonebraker,
in: Hoffmann, Leah: The Path to Clean Data, Communications of the ACM 58(6), p.103, June 2015

SQL DDL: CREATE VIEW

Sicht:
„Virtuelle Tabelle“, d.h. eine Struktur, die selbst keine Daten beinhaltet, sondern deren zugehörige Daten das DBMS immer aus Daten in den Tabellen der DB „ableiten“ bzw. „berechnen“ kann

- **Wird definiert basierend auf bestehenden Tabellen** (oder anderen Sichten …)
  → Tabellen, auf denen eine Sicht aufbaut, werden auch **Basistabellen** genannt

- **Zur Sicht existieren physisch keine eigenen, separaten Daten in der Datenbank**
  - Bei Abfragen der Sicht werden die Daten anhand der Basistabellen, auf denen die Sicht basiert, ermittelt und ausgegeben → „virtuell“
  - Abhängig vom DBMS können die Dateninhalte der Sicht optional auch physisch abgelegt werden (dies wird als **materialisierte Sicht** bezeichnet)-
    → Vor-/Nachteile beachten bzw. abwägen! Im Allg. keine schnellere Antwortzeit durch Nutzung Sicht!

ACHTUNG: Die Materialisierung einer Sicht ist ein rein technisch motivierter Schritt zur Beschleunigung von Anfragen an die Sicht!

SQL DDL: CREATE VIEW

Sicht (Fortsetzung):

- **SQL-Anweisung zum Anlagen einer Sicht: `CREATE VIEW AS` …**
- **Dabei werden Struktur und (virtueller) Inhalt einer Sicht durch eine `SELECT`-Abfrage an die Basistabelle(n) definiert**

  → Sicht(name) kann als „Aliasname“ für eine ggfs. umfangreiche Abfrage verstanden werden

- **Vorteil/Nutzen von Sichten:**
  - **Vereinfachung von Anfragen**
  - **Zugriffskontrolle bzw. Steuerung der Sichtbarkeit von Spalten der Basistabellen**
  - **NICHT: Zur „Beschleunigung“ von Anfragen!**
- **Eine Sicht kann in SQL-Anfragen wie eine Tabelle verwendet werden (wird aber bei der Anfrageabarbeitung vom DBMS unterschiedlich behandelt)**

Vgl. auch externe Ebene der ANSI/SPARC-Architektur (Kap. 2)

→ Die externe Ebene kann z.B. mittels CREATE VIEW <u>im</u> DBS realisiert werden

SQL DDL: CREATE VIEW

Erstellen einer Sicht:

```
CREATE VIEW <Sichtname> AS <Anfrage> ;
```

<Anfrage>: Im Prinzip (fast) beliebige SQL-Anfrage (d.h. SELECT-Anweisung)

Beispiel:

```
CREATE VIEW Stammkunden AS
   SELECT * FROM Kunde WHERE Kundenstatus ="S";
```

(Sehr) einfaches Beispiel zur Verwendung einer Sicht in Anfragen:

```
SELECT Name, Vorname FROM Stammkunden WHERE Ort ="Nürnberg";
```

Was macht das DBMS bei der Anfragebearbeitung daraus? („Ersetzung des Alias“)

```
SELECT Name, Vorname FROM Kunde WHERE Ort ="Nürnberg" AND Kundenstatus ="S";
```

Hinweis:

- Zu Definition und Details der SELECT-Anweisung siehe nachfolgenden Abschnitt zur DML.
- Datenänderungen über die Sicht müssen zu den Basistabellen „passen“ (Datenkonsistenz!)
- Wie bei CREATE TABLE gibt es auch für Sichten Änderungs- und Löschanweisungen: ALTER / DROP VIEW

SQL DDL: CREATE INDEX

Index – *warum?*

- **Häufiger Zugriff auf / Suche nach Daten über bestimmte Spalten (oder Kombinationen von Spalten)**
- **Große Tabelle, d.h. sehr viele Zeilen**

Beschleunigung des Datenzugriffs über die jeweiligen Spalten durch einen Index

- **Da pro Tabelle mehrere solcher Suchbedingungen existieren können, können auch mehrere Indexe für eine Tabelle definiert werden.**

- **Beispiel: Tabelle Kunde**

  Zugriffe auf Kundendaten erfolgen im Unternehmen über Kundennummer, aber auch über das Geburtsdatum, über die Kombination Nachname+Vorname und über die Kombination Nachname + Wohnort

Erstellen von drei Indexen zur Tabelle Kunde

SQL DDL: CREATE INDEX

Index – *was ist das?*

- **Vereinfachte Darstellung: *Physische Struktur*, die die Werte der angegebenen Spalten in sortierter oder „schnell zugreifbarer“ Form beinhaltet und damit einen schnelleren Zugriff auf die Zeilen der zugrundeliegenden Tabelle ermöglicht.**
- **Zu jedem Spaltenwert (bzw. Wertekombination) der „indizierten“ Spalten ist im Index ein *Verweis auf die Zeile* in der zugrundeliegenden Tabelle angegeben.**
  → Ein Index kann als spezielle „Tabelle“ angesehen werden.
- **Üblicherweise werden die Zeilen einer Tabelle automatisch anhand des Primärschlüssel sortiert, d.h. die Angabe des Primärschlüssels hat im DBMS implizit meist die Anlage eines Index zur Folge**
- **Im Prinzip führt jede Schlüsselangabe in einer Tabellendefinition zu einem Index**

Indexe bzw. Indexstrukturen sind Teil des internen Schemas (vgl. interne/physische Ebene der ANSI / SPARC-Architektur, Kap. 2)

Index „Kunde nach Name“:

Name	"Verweis"
…	
Ammermann	
Bach	
Berger	
Müller	
Unterberg	
Zillner	
…	

Tabelle Kunde (Primärschlüssel KdNr):

Kundennr	Name	GebDat	…
…		…	…
1002	Müller	…	…
1006	Ammermann	…	…
1067	Zillner	…	…
2045	Unterberg	…	…
5063	Berger	…	…
7045	Bach	…	…
…		…	…

SQL DDL: CREATE INDEX

Erstellen eines Index:

```
CREATE [UNIQUE] INDEX <Indexname> [USING <Indexart>]
        ON <Tabellenname> ( <Indexspaltenname> {, <Indexspaltenname>}* ) ;
```

<Indexart>: Gibt die physische Speicherstruktur für den Index an,
z.B. BTree oder Hash. (Systemabhängig!)

<Indexspaltenname> := <Spaltenname> [ASC | DESC]

Beispiel:

```
CREATE INDEX Kundenname
    ON Kunde (Name);
```

Zugrundeliegende Tabelle für Beispiel:
Kunde mit Attributen Kundennr (=Primärschlüssel), Name, Vorname, Wohnort, …

Hinweis:

- Die Option `UNIQUE` erzwingt, dass die Werte(kombinationen) im Index eindeutig sind, d.h. nur einmal auftreten.
- Die Angabe `ASC` bzw. `DESC` legt die Sortierreihenfolge für die Werte der Spalte fest (`ASC` → ascending = aufsteigend; `DESC` → descending = absteigend)

SQL DDL: CREATE INDEX

- **Wichtig: Indexe sollten immer auf Spalten mit <u>hoher Selektivität</u> (d.h. viele unterschiedliche Werte) definiert werden**
  - **Ein Index für ein Attribut „Geschlecht“ oder für ein Wahrheitswert-Attribut macht wenig Sinn …**
  - **Für ein Attribut „Nachname“ oder die Kombination „Nachname“ + „Vorname“ hingegen kann ein Index sinnvoll sein, da auf diesen Attributen (relativ) wenige doppelte Werte in einer Tabelle auftreten**

- **Achtung:**
  Bei <u>jeder</u> Aktualisierung einer Tabelle (Einfügen, Ändern, Löschen einer Zeile) müssen <u>alle</u> Indexe von der Änderung betroffenen Indexe der Tabelle ebenfalls aktualisiert werden!

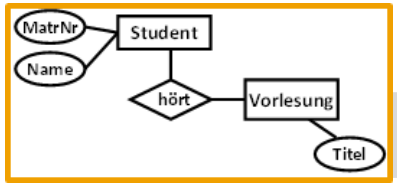
  Indexe <u>beschleunigen</u> somit <u>Lesezugriffe</u> (Abfragen),
  <u>verlangsamen</u> aber <u>Schreibzugriffe</u> (Einfügen, Ändern, Löschen) !

- **Existiert zu einer Anfrage ein passender Index, so wird er vom DBMS automatisch für die Anfragebearbeitung genutzt.**
  → Ein DB-Nutzer oder Anwendungsprogrammierer muss sich nicht um die Verwendung eines Index nicht kümmern (siehe auch Kap. 2, Thema Datenunabhängigkeit).

Bezug zur Drei-Schichten-Architektur aus Kapitel 2: Schema-Implementierung mit SQL

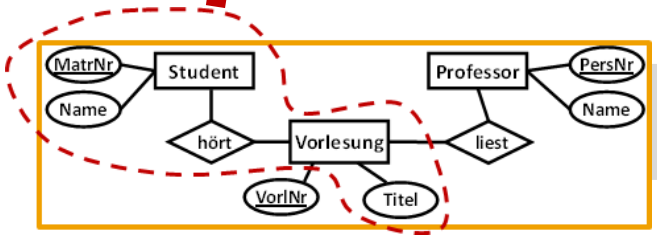
Fachliche Betrachtungsweise:

Korrespondierende SQL-DDL-Anweisungen zur Implementierung des Schemas



Externes Schema (Sicht) 1 ... Externes Schema (Sicht) n

```
CREATE VIEW
DROP VIEW
```

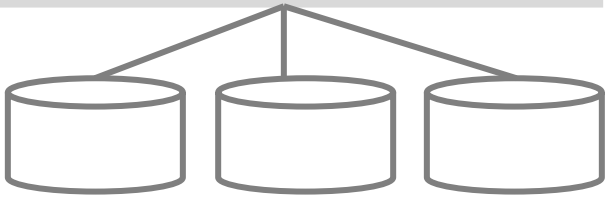


Konzeptionelles Schema

```
CREATE SCHEMA
DROP SCHEMA
CREATE TABLE
ALTER TABLE
DROP TABLE
```

+ ggfs. entsprechende `DOMAIN`-Anweisungen

Internes Schema



```
CREATE INDEX
ALTER INDEX
DROP INDEX
```

SQL DML: INSERT

Einfügen von einer oder mehreren Zeilen in eine Tabelle:

```
INSERT INTO <Tabellenname> [ ( <Spaltenname> {, <Spaltenname>}* ) ]
VALUES (<Wert> {, <Wert>}* ) { , (<Wert> {, <Wert>}* ) }* ;
```

Beispiele:

Zugrundeliegende Tabelle für Beispiel:
Kunde mit Attributen Kundennr (=Primärschlüssel), Name, Vorname, Wohnort, …

a)
```
INSERT INTO Kunde
VALUES (1001, "Müller", "Andreas", "Hamburg", …),
       (1027, "Riedel", "Sandra", NULL, …);
```

b)
```
INSERT INTO Kunde (Name, Vorname, Kundennr)
VALUES ("Siemer", "Jasmin ", 2108),
       ("Erber", "Gerd ", 3497);
```

Hinweis:

- **Die Spaltennamen müssen nicht angegeben werden, vgl. Beispiel a). Fehlt die Angabe jedoch, so ist sicherzustellen, dass die Werte in der Reihenfolge der Spalten in der Tabelle angegeben werden, da ansonsten falsche Zuordnungen oder Fehler aufgrund nicht passender Datentypen auftreten können. Insbesondere müssen dann für <u>alle</u> Spalten Werte angegeben werden**
- **Bei Angabe der Spaltennamen werden nicht aufgeführte Spalten automatisch mit dem Vorgabewert (DEFAULT) bzw. Nullwert (NULL) belegt.**

SQL DML: UPDATE

Ändern von (vorhandenen) Zeilen in einer Tabelle:

```
UPDATE <Tabellenname> SET <Spaltenname> = {<Wert> | <Ausdruck>}
                          {, <Spaltenname> = {<Wert> | <Ausdruck>} }*
[WHERE <Selektionsbedingung>];
```

<Selektionsbedingung>: Im Prinzip beliebige Selektionsausdrücke, wie für die WHERE-Klausel der SELECT-Anweisung (Details siehe dort)

Beispiele:

Zugrundeliegende Tabelle für Beispiel:
Kunde mit Attributen Kundennr, Name, Vorname, Wohnort Kundenstatus, Rabatt, …

a)
```
UPDATE Kunde
SET Name = "Wellenberg", Wohnort = "Köln"
WHERE Kundennr = 2108;
```

b)
```
UPDATE Kunde
SET Rabatt = Rabatt * 1.1
WHERE Kundenstatus = "S";
```

Erinnerung:
S = Stammkunde

Hinweis:

- Eine UPDATE-Anweisung ohne WHERE-Klausel ändert <u>alle</u> Zeilen der Tabelle, was in der Regel nicht erwünscht (oder beabsichtigt) ist

SQL DML: DELETE

Löschen von vorhandenen Zeilen in einer Tabelle:

```
DELETE FROM <Tabellenname> [WHERE <Selektionsbedingung>];
```

<Selektionsbedingung>: Im Prinzip beliebigeSelektionsausdrücke, wie für die WHERE-Klausel der SELECT-Anweisung (Details siehe dort)

Beispiele:

a)

```
DELETE FROM Kunde
WHERE Kundennr = 2108;
```

b)

```
DELETE FROM Kunde
WHERE Kundenstatus = "G";
```

Erinnerung:
G = Gelegenheitskunde

Hinweis:

- **Eine DELETE-Anweisung ohne WHERE-Klausel löscht <u>alle</u> Zeilen der Tabelle**

SQL DML: Datenbankabfragen

SQL bietet eine zentrale Anweisung für die Abfrage von Datenbanken

➡ **Die SELECT-Anweisung**

- **Teil der SQL DML, obwohl keine echte Datenmanipulation (im Sinne von Änderung), sondern lesender Zugriff auf eine oder mehrere Tabellen und Anzeige des Abfrageergebnisses**
  - **Ein Abfrageergebnis ist eine Auswahl und „Aufbereitung“ von Daten der Datenbank gemäß der Abfragekriterien**
  - **Die Abfrage hat keinerlei Änderungen der Datenbank zur Folge**

- **Das Ergebnis einer SELECT-Anweisung kann als „virtuelle Tabelle“ (d.h. Menge von Zeilen) verstanden werden, die dem Nutzer angezeigt bzw. an das Anwendungsprogramm, aus welchem die Anfrage gestellt wurde, übergeben werden.**

- **Hinweise:**
  - **Die SELECT-Anweisung bietet viel mehr (und umfangreichere) Formulierungsmöglichkeiten für Anfragen als die Selektionsoperation der relationalen Algebra**
  - **Die SELECT-Anweisung in SQL deckt mehrere Abfrage-Operationen der relationalen Algebra ab (Selektion, Projektion, Verbund, …)**

SQL DML: Die SELECT-Anweisung im Überblick

Einfache und (fast) vollständige Grundstruktur der SELECT-Anweisung:

```
SELECT [ DISTINCT | ALL ] <Spaltenausdrucksliste>
FROM <Tabellenangabeliste>
[ WHERE <Selektionsbedingung> ]
[ GROUP BY <Spaltenausdrucksliste> ]
[ HAVING <Selektionsbedingung> ]
[ ORDER BY <Spaltenausdrucksliste > [ASC | DESC] ] ;
```

Grundstruktur der SELECT-Anweisung
(„SPJ-Struktur“: Selektion, Projektion, Join)

Gruppierung & Verdichtung / Auswertung von Zeilen im Abfrageergebnis

Sortierung des Abfrageergebnisses

Die SELECT-Anweisung setzt sich aus mehreren Bestandteilen (auch als „Klauseln“ bezeichnet) zusammen

SQL DML: Die SELECT-Anweisung im Überblick

Festlegung von Struktur und Inhalt des Abfrageergebnisses durch die einzelnen Bestandteile der SELECT-Anweisung:

1. `SELECT [ DISTINCT | ALL ]` <Spaltenausdrucksliste> → **Auswahl der Spalten** (Algebra: Projektion)
2. `FROM` <Tabellenangabeliste> → **Auswahl der Tabellen** (mehrere Tabellen: Join oder Kreuzprodukt)
3. `[ WHERE` <Selektionsbedingung> `]` → **Auswahl der Zeilen** (Algebra: Selektion)
4. `[ GROUP BY` <Spaltenausdrucksliste> `]` → **Verdichtung von gruppierten Zeilen zu einer „Gruppenzeile“** (mit „Berechnung“ verbunden!)
5. `[ HAVING` <Selektionsbedingung> `]` → **Angabe von Bedingungen für die Gruppenzeile**
6. `[ ORDER BY` <Sortierliste> `] ;` → **Festlegung der Sortierung des Abfrageergebnisses**

SQL DML: SELECT
Syntax für Grundstruktur der SELECT-Anweisung

```
SELECT [ DISTINCT | ALL ] <Spaltenangabeliste>
FROM <Tabellenangabeliste>
[ WHERE <Selektionsbedingung> ]

mit     <Spaltenangabeliste> := * | <Spaltenangabe> {,<Spaltenangabe>}*

        <Spaltenangabe>      := <Spaltenname> | <Spaltenname> [AS] <Aliasname>
                                | <Tabellenname> . <Spaltenname> | <Tabellenname> . *
                                | <Spaltenfunktion> | <Spaltenausdruck>

        <Tabellenangabeliste> := <Tabellenangabe> {,< Tabellenangabe>}* | <Joinliste>

        <Tabellenangabe> := <Tabellenname> | <Tabellenname> [AS] <Aliasname>
```

SQL DML: SELECT
Grundsyntax der einfachen SELECT-Anweisung

Erläuterungen zur Grundsyntax:

- *** als <Spaltenangabeliste>:**
  Kurzschreibweise für „alle Spalten“ der Tabellen aus <Tabellenangabeliste>
  → erspart die explizite Aufzählung aller Spaltennamen

- **<Spaltenname>: Name einer Spalte einer Tabelle(nstruktur) des zugrundeliegenden Schemas**

- **<Tabellenname>: Name einer Tabelle(nstruktur) des zugrundeliegenden Schemas**

- **Bezieht sich die Abfrage auf Tabellen, die Spalten mit gleichem Namen besitzen, so ist bei der Angabe der Spalte auch der Tabellenname anzugeben.**
  Dieser wird dem Spaltennamen vorangestellt, getrennt durch einen Punkt:
  <Tabellenname>.<Spaltenname>

  (z.B. Firma.Name und Mitarbeiter.Name)

SQL DML: SELECT
Beispiele für SELECT-Klausel

Beispiele:

Zugrunde gelegt für Beispiel:
Tabelle *Person* mit Spalten *Nachname*, *Vorname*, *PLZ*, *Ort*, …

a)
```
SELECT *
FROM Person ;
```

b)
```
SELECT Person.*
FROM Person ;
```

c)
```
SELECT DISTINCT *
FROM Person ;
```

d)
```
SELECT Vorname, Nachname
FROM Person ;
```

e)
```
SELECT DISTINCT Vorname, Nachname
FROM Person ;
```

f)
```
SELECT Person.Vorname, Person.Nachname
FROM Person ;
```

g)
```
SELECT p.Vorname, p.Nachname
FROM Person p ;
```

h)
```
SELECT p.Vorname, p.Nachname
FROM Person AS p ;
```

SQL DML: SELECT
Beispiele für SELECT-Klausel

Beispiele:

Zugrunde gelegt für Beispiel:
Tabelle *Person* mit Spalten *Nachname*, *Vorname*, *PLZ*, *Ort*, …

a) `SELECT` *Vorname, Nachname, PLZ, Ort*
`FROM` *Person* ;

b) `SELECT` *Vorname, Nachname, PLZ* `AS` "*Postleitzahl*", *Ort AS* "*Wohnort der Person*"
`FROM` *Person* ;

Tabelle *Person*

Nachname	Vorname	PLZ	Ort	….
…	…	…	…	…

a)

Struktur des Anfrageergebnisses zu a)

Vorname	Nachname	PLZ	Ort
…	…	…	…

b)

Struktur des Anfrageergebnisses zu b)

Vorname	Nachname	Postleitzahl	Wohnort der Person
…	…	…	…

SQL DML: SELECT
Ausdrücke in der SELECT-Klausel

- **Neben der einfachen Angabe oder Umbenennung von Spalten(namen): Auch Anwendung von Ausdrücken in der SELECT-Klausel möglich.**
  - **Ausdruck: „Beliebige“ (sinnvolle, also insbesondere typkonforme …) Kombination von Spalten(namen), Konstanten, Operatoren und Funktionen**
  - **Generell gilt:**
    An (fast) jeder Stelle, an der laut SQL-Syntax ein Spaltenname stehen darf, ist auch ein Ausdruck zulässig

- **Operatoren**
  - **Passend zu den Datentypen der Spalten**
  - **Numerische Operatoren (+, - , *, /), Datums-/Zeitoperatoren, Zeichenketten-Operatoren**

SQL DML: SELECT
Ausdrücke in der SELECT-Klausel

- **Funktionen**
  - **Skalarfunktionen (zeilen- bzw. einzelwertorientierte Funktionen)**
    - Führen Berechnungen auf einer Zeile bzw. deren Werten durch
  - **Aggregatfunktionen (gruppenorientierte Funktionen)**
    - Führen Berechnungen auf einer Menge (Gruppe) von Zeilen durch, in der Regel basierend auf einer Spalte
  - **Die Anwendung von Operatoren und Funktionen führt i.d.R. zu Spalten im Abfrageergebnis, die in keiner Tabelle der Datenbank vorhanden sind (→ berechnete / abgeleitete Werte)**

SQL DML: SELECT
Funktionen in der SELECT-Klausel

Einige Beispiele für Skalarfunktionen (zeilenorientierte Funktionen)

Numerische Funktionen

- ROUND(m,n) → rundet Zahl m auf n Nachkommastellen
- ABSOLUTE(n) → Absoluter Wert von n (Entfernung Vorzeichen)
- …

Alphanumerische Funktionen

- CONCATENATE (x,y) → Zeichenketten x und y werden zusammengefügt
- SUBSTRING(x, n1, n2) → Ermittelt in der Zeichenkette x ab Position n1 die folgenden n2 Zeichen
- LOWER(x) → Wandelt alle Zeichen der Zeichenkette x in Kleinbuchstaben um
- …

Datums-/Zeitfunktionen

- CURRENT_DATE → Aktuelles Datum
- CURRENT_TIME → Aktuelle Uhrzeit
- CURRENT_TIMESTAMP → aktueller Zeitstempel (Datum + Uhrzeit)
- ...

SQL DML: SELECT
Funktionen in der SELECT-Klausel

Grundsyntax für Aggregatfunktionen:

<Funktionsname> ([ALL|DISTINCT] <X>)

mit <X> Spalte oder einfache Berechnung mit Operatoren/Skalarfunktionen

Option ALL: Aggregatfunktion wird auf alle Werte der Spalte <X> angewandt.
Option DISTINCT: Doppelte Werte der Spalte <X> werden bei der Berechnung der Aggregatfunktion nicht berücksichtigt.

Einige Beispiele für Aggregatfunktionen:

- **Zählen von Zeilen**
  - COUNT(*) → Anzahl der Zeilen (einer Tabelle)
  - COUNT(<Spaltenname>) → Anzahl der Werte ungleich NULL in den Zeilen
- **SUM(< Spaltenname>) → Summenbildung** (nur auf arithmetische Spalten anwendbar)
- **AVG(< Spaltenname>) → Durchschnittswert** (nur auf arithmetische Spalten anwendbar)
- **MIN(< Spaltenname>), MAX(< Spaltenname>) → minimaler bzw. maximaler Wert**
- **…..**

Aggregatfunktionen liefern genau einen Wert und werden meist zusammen mit Gruppierungen verwendet (→ GROUP BY – Klausel der SELECT-Anweisung).

SQL DML: SELECT
Beispiele für Ausdrücke in der SELECT-Klausel

Zugrunde gelegt für Beispiel: Tabelle *Artikel* mit Spalten *ArtNr*, *Bezeichung*, *Nettopreis*, *Gewicht*, …
Tabelle *Person* mit Spalten *Nachname*, *Vorname*, *PLZ*, *Ort*, …

a) `SELECT ROUND(`*Nettopreis * 1.19 , 2)* `AS` *Bruttopreis*
`FROM` *Artikel ;*

→ Verwendung Operator und Skalarfunktion

b) `SELECT COUNT(*)`
`FROM` *Artikel ;*

(Anzahl der Zeilen in Artikel)

→ Verwendung Aggregatfunktion

c) `SELECT COUNT(`*Gewicht*`)`
`FROM` *Artikel ;*

(Anzahl der Gewichtswerte (ungleich NULL) in Artikel)

→ Verwendung Aggregatfunktion

d) `SELECT COUNT(DISTINCT` *Vorname*`)`
`FROM` *Person ;*

(Anzahl der verschiedenen Vornamen in Person)

→ Verwendung Aggregatfunktion

e) `SELECT AVG(`*Nettopreis*`) AS "`*Durchschnittlicher Nettopreis*`"`
`FROM` *Artikel ;*

→ Verwendung Aggregatfunktion

SQL DML: SELECT
Selektionsbedingungen in der WHERE-Klausel

<Selektionsbedingung> in WHERE-Klausel

- **Festlegung einer logischen Bedingung, die die Zeilen des Anfrageergebnisses erfüllen müssen**

- **Die Bedingung kann beinhalten:**
  - **Die bekannten Vergleichsoperatoren `=`, `<>`, `<=`, `>=`, `<`, `>`**
  - **Die logischen Operatoren NOT, AND, OR**
  - **Operator zur Prüfung auf den Nullwert: `IS [NOT] NULL`**
  - **Operator für Vergleiche mit Mustern: `[NOT] LIKE`**
    - **% als Platzhalter für eine beliebige Zeichenkette (Achtung: nicht `*` !)**
    - **_ als Platzhalter für genau ein beliebiges Zeichen (Achtung: nicht `?` !)**
  - **Intervalloperator BETWEEN**
    - **Gibt ein Werteintervall an**
    - **Angegebene Grenzen gehören zum gültigen Intervall**
    - **Kann auch durch obige Vergleichsoperatoren abgebildet werden**

SQL DML: SELECT
Selektionsbedingungen in der WHERE-Klausel

Beispiele

- *Ort* = "*Hamburg*"
- *Vorname* = "*Frank*" AND *Name* = "*Meyer*"
- *Geburtsdatum* `IS NOT NULL`
- *Vorname* `IS NULL`
- *PLZ* `LIKE` "*68%*"
- *PLZ* LIKE "*68_ _0*"
- *Ort* `LIKE` "*%stadt*"
- *PLZ* `BETWEEN` *20000* AND *20999*

SQL DML: SELECT
Selektionsbedingungen in der WHERE-Klausel

Auswertung der Selektionsbedingung in der WHERE-Klausel:

- **Da Zeilen Nullwerte beinhalten können, wird bei der Auswertung der Bedingung die dreiwertige Logik mit den Wahrheitswerten TRUE, FALSE, UNKNOWN verwendet**
  - **a) Jeder Vergleich mit einem Nullwert ergibt den Wahrheitswert UNKNOWN**
    (Achtung: UNKNOWN ist nicht NULL !)
  - **b) Alle logischen Operatoren werden mit dreiwertiger Logik ausgewertet**
    → Wahrheitstabellen siehe Tafelmitschrift
  - **c) Es werden diejenigen Zeilen in das Anfrageergebnis übernommen (ausgegeben), für die die logische Bedingung zu „wahr“ (TRUE) ausgewertet wird.**

Prioritäten / Bindungsstärke der Operatoren bei der Auswertung:

- **Erst Vergleichsoperatoren**
- **Dann logische Operatoren in der Reihenfolge NOT, AND, OR**

Andere Auswertungsreihenfolge gewünscht? Dann entsprechende Klammerung nötig …
(wie allgemein von logischen bzw. arithmetischen Ausdrücken bekannt.)

SQL DML: SELECT
Beispiele für einfache SELECT-FROM-WHERE - Anweisungen

```
SELECT ROUND (Nettopreis * 1.19 , 2) AS Bruttopreis
FROM Artikel
WHERE Nettopreis > 100 ;
```

Bruttopreis aller Artikel,
die mehr als 100 kosten

```
SELECT *
FROM Person
WHERE GebDatum = 24.12.1985 AND Vorname IS NOT NULL;
```

Personendaten der Personen,
die am 24.12.1985 geboren sind
und für die der Vorname bekannt ist

```
SELECT Vorname, Name, GebDatum
FROM Angestellter
WHERE (Gehalt BETWEEN 50000 AND 75000) AND Standort = "Stuttgart" ;
```

Vorname, Name und Geburtsdatum aller Angestellten,
die am Standort Stuttgart zwischen 50000 und 75000 verdienen

```
SELECT COUNT(*)
FROM Ausleihe
```

Anzahl Ausleihen
(bei der betrachteten Bedeutung von Ausleihe also:
Anzahl der aktuell ausgeliehenen Bücher)

SQL DML: SELECT
Verknüpfung von Tabellen

Bisher wurden nur einzelne Tabellen betrachtet …

- **Abfragen und Auswertungen über mehrere Tabellen?**
- **Wie werden Join(s) und Kreuzprodukt in SQL formuliert?**

Angabe mehrerer Tabellen in <Tabellenangabeliste> (bzw. <Joinliste>) der FROM-Klausel, sowie entsprechender Bedingungen zwischen den Spalten

- **Vor SQL92: Darstellung von Joins allein durch Angabe von mehreren Tabellen und Spaltenvergleichen in der WHERE-Klausel:**

Annahme:
A ist die Spalte,
über die der Join erfolgt

… `FROM` *Tabelle1*, *Tabelle2*
`WHERE` *Tabelle1.A = Tabelle2.A*

- **Ab SQL92: Angabe von Joins durch Schlüsselwort in der FROM-Klausel möglich (z.B. `NATURAL JOIN`):**

… `FROM` *Tabelle1* `NATURAL JOIN` *Tabelle2*

Klare Trennung: Join-Bedingung in der FROM-Klausel.
In der WHERE-Klausel finden sich nur noch Selektionsbedingungen, die nicht der Verknüpfung der Tabellen dienen

SQL DML: SELECT
Verknüpfung von Tabellen: Beispiel

Join von Leser und Ausleihe über LeserNr (in alter Notation, d.h. vor SQL92):

a)
```
SELECT L.Name AS Lesername, A.InvNr, A.Datum AS Ausleihdatum
FROM Leser L, Ausleihe A
WHERE L.LeserNr = A.LeserNr ;
```

Join von Leser und Ausleihe über LeserNr mit NATURAL JOIN in From-Klausel:

b)
```
SELECT L.Name AS Lesername, A.InvNr, A.Datum AS Ausleihdatum
FROM Leser L NATURAL JOIN Ausleihe A ;
```

Zur Erinnerung hier die Struktur der zugrundeliegenden Tabellen:

Ausleihe (InvNr, LeserNr, Datum)

Leser (LeserNr, Name, Telefon)

SQL DML: SELECT
Verknüpfung von mehr als 2 Tabellen: Beispiel

In gleicher Weise können auch mehrere Tabellen verknüpft werden.

Beispiel mit den bekannten Tabellen Buch, Ausleihe, Leser:

a)
```
SELECT  L.Name AS Lesername, B.Titel AS Buchtitel, A.Datum AS Ausleihdatum
FROM    Buch B, Ausleihe A, Leser L
WHERE   B.InvNr = A.InvNr AND A.LeserNr = L.LeserNr ;
```

b)
```
SELECT  L.Name AS Lesername, B.Titel AS Buchtitel, A.Datum AS Ausleihdatum
FROM    Buch B NATURAL JOIN
          (Ausleihe A NATURAL JOIN Leser L) ;
```

Zur Erinnerung hier die Struktur der zugrundeliegenden Tabellen:

Ausleihe (InvNr, LeserNr, Datum)

Leser (LeserNr, Name, Telefon)

Buch (InvNr, Autor, Titel, Verlag, Jahr)

SQL DML: SELECT
Verknüpfung von Tabellen: Überblick

Die neueren Versionen des SQL-Standards unterstützen verschiedene Verbundoperationen der Relationenalgebra durch eigene Schlüsselworte (und damit separate SQL-Operationen). Im Wesentlichen sind dies:

- `NATURAL JOIN` **(entspricht dem natürlichen Verbund)**
- `INNER JOIN` **(entspricht dem allgemeinen Join (Θ-Join), flexibelste Verbundoperation)**
- `LEFT OUTER JOIN / RIGHT OUTER JOIN / FULL OUTER JOIN`
  (entspricht den 3 Outer Join Operationen der Algebra
  … daneben gibt es auch noch die Varianten `NATURAL LEFT / RIGHT / FULL OUTER JOIN`)
- `CROSS JOIN` **(Kreuzprodukt)**

Bedeutung der Operationen? Analog zu den Algebra-Operationen:

`JOIN`

(und der Vollständigkeit halber…:)

`NATURAL`	`INNER`	… `OUTER`	… `NATURAL` … `OUTER`
Vordefinierte Verknüpfung der Zeilen (hinsichtlich der zum Vergleich verwende-tenSpalten und des Vergleichsoperators)	Frei definierbare Verknüpfung der Zeilen (→ explizite Auswahl der Spalten für Vergleich und des Vergleichsoperators)	Frei definierbare Verknüpfung der Zeilen (→ siehe INNER), zusätzlich werden nicht verknüpfbare Zeilen aus linker, rechter oder beiden Tabellen übernommen	Vordefinierte Verknüpfung der Zeilen (→ siehe NATURAL), zusätzlich werden nicht verknüpfbare Zeilen aus linker, rechter oder beiden Tabellen übernommen

SQL DML: SELECT
Verknüpfung von Tabellen: Syntax

Um in einer SQL-Anfrage Join-Ausdrücke nutzen zu können, wird die <Tabellenangabeliste> in der `FROM`–Klausel durch eine <Joinliste> ersetzt und so erweitert.

Grundsyntax für <Joinliste>:

```
<Joinliste>        := <Joinausdruck> <Joinoperator> <Joinausdruck> [ ON <Joinbedingung>]

<Joinausdruck>     := <Tabellenangabe> | <Joinliste>

<Joinoperator>     := NATURAL JOIN
                    | NATURAL [ LEFT|RIGHT|FULL ] OUTER JOIN
                    | [INNER] JOIN
                    | [ LEFT|RIGHT|FULL ] OUTER JOIN
                    | CROSS JOIN

<Tabellenangabe> := <Tabellenname> | <Tabellenname> [AS] <Aliasname>
```

Bedeutung der Join-Operatoren: Vgl. entsprechende Algebra-Operationen, Kap.4

SQL DML: SELECT
Verknüpfung von Tabellen: Syntax

Erläuterungen zu <Joinliste>:

- **Die <Joinbedingung> besteht aus einem oder mehreren Vergleichen, verknüpft mit AND**
- **Der Anweisungsteil „ON <Joinbedingung>“ darf bei NATURAL JOIN bzw. NATURAL …OUTER JOIN nicht angegeben werden.**
  Bei den übrigen Join-Anweisungen ist er optional.
- **Es können (im Prinzip…) beliebig viele Tabellen mittels der <Joinliste> verknüpft werden.**
- **Es kann sinnvoll sein (aus Gründen der Übersichtlichkeit und ggfs. zur Festlegung einer Abarbeitungsreihenfolge durch das DBMS), die einzelnen Joinausdrücke in der Liste vollständig zu klammern**
  (die Angabe von Klammern ist in obiger Syntax nicht aufgeführt)
- **Sind keine Klammern angegeben, wird die <Joinliste> durch das DBMS im Prinzip von links nach rechts abgearbeitet.**
  (vorbehaltlich Änderungen im Rahmen der Anfrageoptimierung durch das DBMS)

SQL DML: SELECT
Verknüpfung von Tabellen

Auch in SQL kann der natürliche Verbund mittels des allgemeineren Θ-Joins dargestellt werden. Im Unterschied zum NATURAL JOIN ist dann beim INNER JOIN

- **die Join-Bedingung explizit im „ON“-Teil der Anweisung anzugeben**
- **insbesondere für Spalten, die in mehreren Tabellen vorkommen, in der SELECT-Klausel festzulegen, aus welcher Tabelle die Spalte in das Anfrageergebnis übernommen werden soll**

Beispiel:

Join von Leser und Ausleihe über LeserNr (als natürlicher Verbund):

a)
```
SELECT L.Name AS Lesername, A.InvNr, A.Datum AS Ausleihdatum
FROM Leser L NATURAL JOIN Ausleihe A;
```

Join von Leser und Ausleihe über LeserNr (als allgemeiner Verbund):

b)
```
SELECT L.Name AS Lesername, A.InvNr, A.Datum AS Ausleihdatum
FROM Leser L INNER JOIN Ausleihe A ON (L.LeserNr = A.LeserNr);
```

SQL DML: SELECT
Verknüpfung von mehr als 2 Tabellen

In gleicher Weise können wie beim NATURAL JOIN mittels INNER JOIN auch mehrere Tabellen verknüpft werden.

Beispiel mit den bekannten Tabellen *Buch*, *Ausleihe*, *Leser*:

```
SELECT  L.Name AS Lesername, B.Titel AS Buchtitel, A.Datum AS Ausleihdatum
FROM    Buch B INNER JOIN
          (Ausleihe A INNER JOIN Leser L ON (A.LeserNr = L.LeserNr ))
          ON B.InvNr = A.InvNr ;
```

Zur Erinnerung hier die Struktur der zugrundeliegenden Tabellen:

Ausleihe (InvNr, LeserNr, Datum)

Leser (LeserNr, Name, Telefon)

Buch (InvNr, Autor, Titel, Verlag, Jahr)

SQL DML: SELECT
Verknüpfung von mehr als 2 Tabellen

Hinweise zu den Join-Anweisungen in SQL:

- **Die Kreuzproduktbildung (CROSS JOIN in SQL) kann auch durch Weglassen von „ON <Joinbedingung>“ in einem INNER JOIN erreicht werden**

- **Θ-Join (INNER JOIN in SQL): Verwendung anderer Operatoren (z.B. „<“) statt „=“ in der <Joinbedingung> möglich.**
  - **Somit können die Werte in namensgleichen Spalten der beiden Tabellen verschieden sein, auch wenn sie die Bedingung erfüllen**
  - **Daher kann es im Unterschied zum NATURAL JOIN Sinn machen, je nach Bedarf bei namensgleichen Spalten beide Spalten in das Ergebnis eines INNER JOIN zu übernehmen**

- **Mischung der verschiedenen JOIN-Anweisungen in einer SELECT-Anweisung**
  - **Prinzipiell können INNER und OUTER JOINS (und NATURAL JOINS) in der <Joinliste> gemischt werden.**
  - **Dies führt jedoch zu Effekten, die der eigentlichen Zielsetzung der Outer Joins (= Übernahme nicht verbindbarer Zeilen in das Ergebnis) zuwiderlaufen, da durch die Anwendung eines INNER JOINS auf ein Zwischenergebnis, das durch einen OUTER JOIN entstanden ist, in der Regel wieder Zeilen entfallen (u.a. auch aufgrund des Vergleichs mit Nullwerten).**
  - **Daher sollten in einem solchen Fall vor der Durchführung der OUTER JOINS erst alle INNER JOINS durchgeführt werden.**

SQL DML: SELECT
Die verschiedenen JOIN-Anweisungen auf einen Blick

Operation aus Rel. Algebra	SQL-Anweisung *	Struktur des Anfrage-ergebnisses in SQL*
$t1 \bowtie t2$	… FROM t1 NATURAL JOIN t2	A B C
$t1 \bowtie_{<Joinbed.>} t2$	… FROM t1 [INNER] JOIN t2 ON (<Joinbed.>)	A B B C
$t1 ⟕_{<Joinbed.>} t2$	… FROM t1 LEFT OUTER JOIN t2 ON (<Joinbed.>)	
$t1 ⟖_{<Joinbed.>} t2$	… FROM t1 RIGHT OUTER JOIN t2 ON (<Joinbed.>)	
$t1 ⟗_{<Joinbed.>} t2$	… FROM t1 FULL OUTER JOIN t2 ON (<Joinbed.>)	
$t1 ⟕ t2$	… FROM t1 NATURAL LEFT OUTER JOIN t2	A B C
$t1 ⟖ t2$	… FROM t1 NATURAL RIGHT OUTER JOIN t2	
$t1 ⟗ t2$	… FROM t1 NATURAL FULL OUTER JOIN t2	
$t1 \times t2$	… FROM t1 [INNER] JOIN t2 (d.h. ohne Angabe der <Joinbed.>)	A B B C
	… FROM t1 CROSS JOIN t2	

*Ausgangstabellen(typen):

t1

A	B

t2

B	C

Ausserdem können NATURAL JOIN, INNER JOIN und CROSS JOIN auch allein durch Angabe entsprechender Bedingungen in der WHERE-Klausel und Spaltenangaben in der SELECT-Klausel formuliert werden (d.h. ohne JOIN, siehe Erläuterung und Beispiele zuvor)

SQL DML: SELECT
GROUP BY-Klausel

- **In großen Datenbeständen haben Zeilen auf einzelnen Spalten oft gleiche Werte, z.B.**
  - **Kundenstatus (Wichtiger Kunde, Stammkunde, Gelegenheitskunde) in Tabelle Kunde**
  - **Abteilungsnr in Tabelle Mitarbeiter**
  - **…**

Gruppenbildung der Zeilen anhand der Werte einer (oder mehrerer) Spalten

- **Bedarf: Auswertungen (Verdichtung, Aggregation) hinsichtlich dieser Gruppen**
  - **Verdichtung / Aggregation:**
    Anwendung von Funktionen (z.B. Zählen oder Summieren) auf jede der Gruppen
    - **Anzahl Kunden pro Gruppe (→ Anzahl wichtiger Kunden, Anzahl Stammkunden, …)**
    - **Mitarbeiterzahl pro Abteilung**
    - **Durchschnittliches Gehalt der Mitarbeiter pro Abteilung**
  - **<u>Pro Gruppe eine (verdichtete) Zeile</u> als Anfrageergebnis**

die Grundlage hierfür bildet die GROUP BY-Klausel der SELECT-Anweisung

SQL DML: SELECT
GROUP BY-Klausel

Syntax:

```
GROUP BY <Spaltenname> {, <Spaltenname>}*
```

- <Spaltenname> bezeichnet eine Spalte aus den Tabellen der FROM-Klausel, anhand derer die Gruppenbildung erfolgt
- Angabe mehrerer Spalten → Festlegung von Gruppen in der Gruppe (mehrstufige Gruppenbildung), z.B. Stammkunden gruppiert nach Ort

In der SELECT-Klausel wird dabei festgelegt, welche Daten die verdichtete Zeile pro Gruppe enthält

- <u>Genau eine</u> verdichtete Zeile pro Gruppe

  ACHTUNG: Es werden nicht nur Gruppen gebildet,
  sondern es erfolgt *<u>immer</u>* pro Gruppe eine Verdichtung.

- Zusammenspiel GROUP BY-Klausel mit SELECT-Klausel

Beispiel:

```
SELECT     Kundenstatus, COUNT(*) AS "Anzahl Kunden"
FROM       Kunde
GROUP BY   Kundenstatus;
```

Hinweis: Die Spalte aus der GROUP BY-Klausel kann, muß aber nicht in der SELECT-Klausel auftreten

SQL DML: SELECT
GROUP BY-Klausel: Beispiel

Tabelle *Kunde*

KdNr	Nachname	Vorname	Kundenstatus	Ort
450009	Gerber	Emil	G	Köln
450007	Weiher	Anke	W	Hamburg
454587	Merkel	Andrea	S	Konstanz
230065	Jellinek	Frank	W	München
760023	Ammer	Rita	G	Köln
547132	Gertel	Manfred	G	München
119754	Meck	Cathrin	S	Rostock
905423	Holler	Jörg	G	Köln
570723	Quack	Günther	S	Frankfurt

```
SELECT     Kundenstatus, COUNT(*) AS "Anzahl Kunden"
FROM       Kunde
GROUP BY   Kundenstatus;
```

Kundenstatus	Anzahl Kunden
G	4
S	3
W	2

- Spalte *Kundenstatus* als Kriterium der Gruppenbildung
- Pro Wert von *Kundenstatus* in *Kunde* eine aggregierte Zeile im Ergebnis
- Funktion wird <u>pro Gruppe</u>, nicht auf gesamte Tabelle angewandt

SQL DML: SELECT
GROUP BY-Klausel: Beispiel

Tabelle *Kunde*

KdNr	Nachname	Vorname	Kundenstatus	Ort
450009	Gerber	Emil	G	Köln
450007	Weiher	Anke	W	Hamburg
454587	Merkel	Andrea	S	Konstanz
230065	Jellinek	Frank	W	München
760023	Ammer	Rita	G	Köln
547132	Gertel	Manfred	G	München
119754	Meck	Cathrin	S	Rostock
905423	Holler	Jörg	G	Köln
570723	Quack	Günther	S	Frankfurt

```
SELECT     Kundenstatus, Ort, COUNT(*) AS "Anzahl Kunden"
FROM       Kunde
GROUP BY   Kundenstatus, Ort;
```

Kundenstatus	Ort	Anzahl Kunden
G	Köln	3
G	München	1
S	Frankfurt	1
S	Konstanz	1
S	Rostock	1
W	Hamburg	1
W	München	1

SQL DML: SELECT
HAVING-Klausel

HAVING-Klausel

- **Ziel: Angabe von Bedingungen für die Gruppen der GROUP BY-Klausel**
  - Die WHERE-Klausel einer Anfrage wird <u>vor</u> der Gruppenbildung ausgewertet
  - Bedingungen, die sich auf das Ergebnis der Gruppenbildung beziehen, können daher mit WHERE nicht formuliert werden
  - Beispiel: Es sollen nur Kundenstatusgruppen betrachtet werden, die jeweils mindestens 3 Kunden beinhalten

Angabe solcher Bedingungen in der HAVING-Klausel der SELECT-Anfrage
→ „Filterung“ / „Selektion“ des Ergebnisses <u>nach</u> der Gruppenbildung

- **Syntax:**

```
HAVING <Spaltenbedingung> { AND | OR <Spaltenbedingung >}*
```

  – <Spaltenbedingung> ist eine Bedingung für eine der Spalten bzw. einen der Ausdrücke in der SELECT-Klausel. Auch hier besteht somit ein enger Zusammenhang mit der SELECT-Klausel (und der GROUP BY-Klausel)
  – Die Bedingung der HAVING-Klausel wird nach Bildung der verdichteten Zeilen pro Gruppe angewendet. Verdichtete Zeilen, die die Bedingung nicht erfüllen, werden nicht in das Ergebnis übernommen.

SQL DML: SELECT
HAVING-Klausel: Beispiel

Beispiel:

```
SELECT      Kundenstatus, COUNT(*) AS "Anzahl Kunden"
FROM        Kunde
GROUP BY    Kundenstatus
HAVING      COUNT(*) >= 3 ;
```

KdNr	Nachname	Vorname	Kundenstatus	Ort
450009	Gerber	Emil	G	Köln
450007	Weiher	Anke	W	Hamburg
454587	Merkel	Andrea	S	Konstanz
230065	Jellinek	Frank	W	München
760023	Ammer	Rita	G	Köln
547132	Gertel	Manfred	G	München
119754	Meck	Cathrin	S	Rostock
905423	Holler	Jörg	G	Köln
570723	Quack	Günther	S	Frankfurt

Kundenstatus	Anzahl Kunden
G	4
S	3
~~W~~	~~2~~

SQL DML: SELECT
ORDER BY-Klausel

- **Die ORDER BY-Klausel der SELECT-Anweisung legt generell die Sortierreihenfolge des Anfrageergebnisses fest**

- **Syntax:**

```
ORDER BY <Sortierliste>
mit
<Sortierliste>          := <Spaltensortierung> {,<Spaltensortierung>}*
<Spaltensortierung> := <Spaltenname> [ASC | DESC]
```

  - **Dabei ist <Spaltenname> eine der Spaltenangaben aus der SELECT-Klausel der Anfrage**
  - **Sortierangabe: ASC → Ascending (aufsteigend); DESC → Descending (absteigend)**
    Die Auswirkung der Sortierangabe hängt von Datentyp und gewählten Zeichensatz ab!
  - **Ist keine Sortierangabe angegeben, legt das DBMS als Default-Wert ASC zugrunde.**
  - **Statt expliziter Angabe der Spalte kann (als Abkürzung) auch die Position der Spalte in der SELECT-Klausel angegeben werden**

SQL DML: SELECT
ORDER BY-Klausel: Beispiele

a1)
```
SELECT  Titel,  Autor,  Jahr
FROM  Buch
ORDER BY Titel, Jahr ;
```

a2)
```
SELECT  Titel,  Autor,  Jahr
FROM  Buch
ORDER BY Titel  ASC, Jahr  ASC;
```

a3)
```
SELECT  Titel,  Autor,  Jahr
FROM  Buch
ORDER BY 1, 3 ;
```

Alle drei Anfragen liefern dasselbe Ergebnis:

Alle Bücher, aufsteigend sortiert nach Titel. Bücher mit gleichem Titel werden aufsteigend nach Jahr sortiert.

b)
```
SELECT  Titel,  Autor,  Jahr
FROM  Buch
ORDER BY Titel  ASC, Jahr  DESC;
```

Alle Bücher, aufsteigend sortiert nach Titel. Bücher mit gleichem Titel werden absteigend nach Jahr sortiert

SQL DML: SELECT
Geschachtelte Anfragen und Mengenvergleiche

Typische Anforderung:
Formulierung einer Anfrage, in der Werte verwendet werden, die erst durch eine andere Anfrage ermittelt werden müssen.

- **Beispiel:**
  „Welche Kunden wohnen im gleichen Ort wie der Kunde mit der Kundennummer 6712?“
  Anfrage
  Anfrage

- **Lösungsmöglichkeit: Zwei Anfragen ***
  - **Ermittle Wohnort des Kunden 6712:**
    `SELECT` *Wohnort* `FROM` *Kunde* `WHERE` *KdNr = 6712* → z.B. Ergebnis „München“
  - **Stelle zweite Anfrage mit ermittelter Konstante, um das eigentlich gewünschte Ergebnis zu erhalten:**
    `SELECT * FROM` *Kunde* `WHERE` *Wohnort = “München“*

Nachteile: a) DBMS muss zwei Anfragen verarbeiten
b) Zweite Anfrage muss „individuell“ formuliert werden, abhängig vom Ergebnis der ersten Anfrage

- **Stattdessen: Formulierung als sogenannte geschachtelte SELECT-Anfrage ****

```
SELECT * FROM Kunde  WHERE Wohnort =  ( SELECT Wohnort
                                        FROM Kunde WHERE KdNr = 6712)
```

Bezeichnung: „Unteranfrage“

SQL DML: SELECT
Geschachtelte Anfragen und Mengenvergleiche

- **Bei geschachtelte Anfragen mit Vergleichsoperatoren (=, <>, …) sind zwei Bedingungen zu beachten:**
  - **a) Die SELECT-Klausel der Unteranfrage muss genau eine Spalte enthalten**
  - **b) Die Ergebnismenge der Unteranfrage darf nicht mehr als eine Zeile beinhalten**

- **Verdeutlichung am vorherigen Beispiel:**

a) b) (da KdNr Schlüssel ist)

```
SELECT * FROM Kunde  WHERE Wohnort = ( SELECT Wohnort
                                       FROM Kunde  WHERE KdNr = 6712)
```

```
SELECT * FROM Kunde  WHERE Wohnort = ( SELECT *
                                       FROM Kunde  WHERE KdNr = 6712)
```

→ Fehler aufgrund Verletzung von a), ebenso bei Angaben wie SELECT *Wohnort*, *Plz* FROM…

```
SELECT * FROM Kunde  WHERE Wohnort = ( SELECT Wohnort
                                       FROM Kunde  WHERE Name = "Müller" )
```

→ Führt zu (Laufzeit-) Fehler aufgrund Verletzung von b), sobald mehr als ein Kunde mit Namen Müller in der Tabelle vorhanden ist

- **Bei Aggregatfunktion in der Unteranfrage kann b) nicht verletzt werden:**

```
SELECT  ArtNr, ArtBezeichnung
FROM    Artikel
WHERE   Preis = (SELECT MIN(Preis)*2 FROM Artikel ) ;
```

„Artikel deren Preis dem doppelten niedrigsten Preis entspricht“

Vorsicht: *

```
SELECT  ArtNr, ArtBezeichnung
FROM    Artikel
WHERE   Preis = MIN(Preis)*2 ;
```

SQL DML: SELECT
Geschachtelte Anfragen und Mengenvergleiche

- **Neben geschachtelten Anfragen mit Vergleichsoperatoren bietet SQL sogenannte Prädikate an. Diese ermöglichen den Vergleich zwischen einem einzelnen Wert und einer Menge von Werten (bzw. Zeilen) in der WHERE-Klausel.**

- **Die (wichtigsten) Prädikate: ALL, ANY, IN, EXISTS**
  (Vergleiche hierzu auch die bekannten Quantoren aus der Prädikatenlogik)

- **Beispiel zur ALL:**

„Name & Vorname der Mitarbeiter, deren Gehalt höher ist als das Gehalt aller Mitarbeiter in der Einkaufsabteilung“

```
SELECT  Name, Vorname
FROM    Mitarbeiter
WHERE   Gehalt > ALL (SELECT Gehalt
                      FROM   Mitarbeiter
                      WHERE  Abteilung = "Einkauf" ) ;
```

- **Hierbei wird die Bedingung**

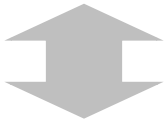
  <Spaltenausdruck> <Vergleichsoperator> ALL (<Unteranfrage>)

  zu WAHR ausgewertet, wenn *für alle* Zeilen Y aus der Ergebnismenge der <Unteranfrage> der Vergleich <Spaltenausdruck> <Vergleichsoperator> Y das Ergebnis „wahr“ liefert.

SQL DML: SELECT
Geschachtelte Anfragen und Mengenvergleiche

Die Anfrage aus dem vorherigen Beispiel kann auch als geschachtelte Anfrage ohne ALL-Prädikat formuliert werden:

```
SELECT  Name, Vorname
FROM    Mitarbeiter
WHERE   Gehalt > ALL (SELECT Gehalt
                      FROM   Mitarbeiter
                      WHERE  Abteilung = "Einkauf" ) ;
```



Alternative geschachtelte Anfragen
mit gleichem Ergebnis

```
SELECT  Name, Vorname
FROM    Mitarbeiter
WHERE   Gehalt > (SELECT MAX(Gehalt )
                  FROM   Mitarbeiter
                  WHERE  Abteilung = "Einkauf" ) ;
```

Oft lässt sich eine natürlichsprachliche Anfrage durch mehr als nur eine SQL-Anfrage formulieren. Dies ist (nicht nur, aber insbesondere) auch bei Verwendung von Anfragen mit Unteranfragen der Fall.

SQL DML: SELECT
Geschachtelte Anfragen und Mengenvergleiche

Beispiel einer geschachtelten Anfrage, die nicht durch Verwendung einer Aggregatfunktion umformuliert werden kann:

„Name & Vorname der Mitarbeiter, die in einem Ort wohnen, in dem kein Mitarbeiter der Einkaufsabteilung wohnt“

```
SELECT  Name, Vorname
FROM    Mitarbeiter
WHERE   Wohnort  <> ALL  (SELECT  Wohnort
                          FROM    Mitarbeiter
                          WHERE   Abteilung  =  " Einkauf " ) ;
```

Unterabfrage nach den Wohnorten der Mitarbeiter in der Einkaufsabteilung

SQL DML: SELECT
Geschachtelte Anfragen und Mengenvergleiche

- **ANY-Prädikat:**
  <Spaltenausdruck> <Vergleichsoperator> `ANY` (<Unteranfrage>):

  Wird zu „wahr“ ausgewertet, wenn *<u>für mindestens eine</u>* Zeile Y aus der Ergebnismenge der <Unteranfrage> der Vergleich „wahr“ ist

  → „Prüfung, ob mindestens eine passende Zeile / Wert in der Ergebnismenge ist“

- **IN-Prädikat:**
  <Spaltenausdruck> `[NOT] IN` (<Unteranfrage>):

  Wird zu „wahr“ ausgewertet, wenn der Wert für <Spaltenausdruck> (nicht) in der Ergebnismenge der <Unteranfrage> ist.

  → „Prüfung, ob eine Zeile / ein Wert in der Ergebnismenge (nicht) enthalten ist“

Beispiel:

```
SELECT  MA-Nr
FROM    Mitarbeiter
WHERE   Mitarbeiter.Projekt-Nr IN (SELECT Projekt-Nr
                                  FROM   Projekt
                                  WHERE  Projektstatus = "kritisch" ) ;
```

Alternative Formulierung: … *Mitarbeiter.Projekt-Nr = ANY (…)*

SQL DML: SELECT
Geschachtelte Anfragen und Mengenvergleiche

- **EXISTS-Prädikat:**

  … `WHERE` `[NOT] EXISTS` (<Unteranfrage>):

  `EXISTS` wird zu „wahr“ ausgewertet, wenn die <Unteranfrage> nicht die leere Menge liefert.

- **Beispiel:**

  „Alle Bestellungen, in denen Artikel 37 bestellt wird“
  (= „Alle Bestellungen, die eine Position mit Artikel 37 beinhalten“)

```
SELECT *
FROM   Bestellung b
WHERE  EXISTS (SELECT *
               FROM  Bestellposition bp
               WHERE  bp.ArtNr = 37  AND  b.BestNr = bp.BestNr ) ;
```

Hinweise:
Diese Anfrage kann auch ohne Unteranfrage formuliert werden (→ wie?).
Die Anfrage beinhaltet eine sogenannte „korrelierte Unteranfrage“ (s.u.).

SQL-Anfragen formulieren

… und wie geht man zielgerichtet / geordnet vor, um zu einer gegebenen natürlichsprachlich formulierten Anfrage eine gleichwertige SQL-Anfrage zu erstellen ???

Anfrageabarbeitung & Anfrageoptimierung in RDBMS: Prinzipieller Ablauf (vereinfachte Darstellung)

SQL-Anfrage (Programm / Anwender)

Korrektheitsprüfung, Sichtauflösung & Übersetzung

Algebraausdruck als Anfrage-/Operatorbaum („Anfrageplan“)

Vereinfachung & Standardisierung

Vereinfachter Anfrageplan

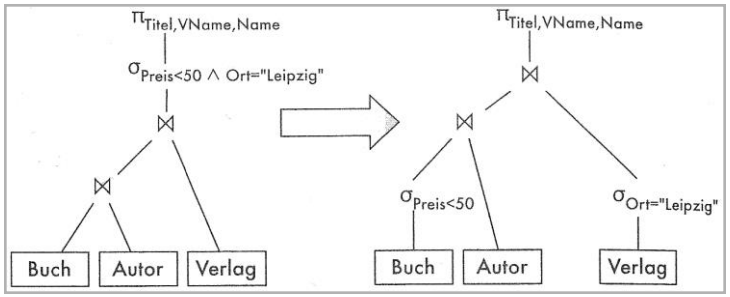
Anfrageoptimierung („Umschreiben“ des Plans)

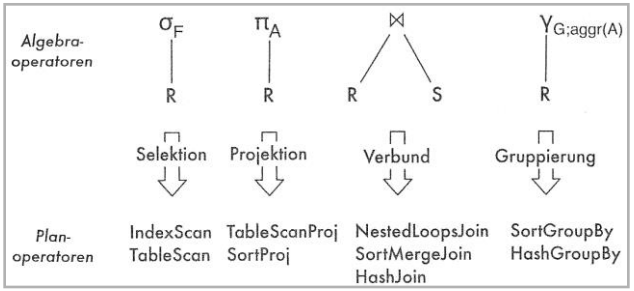
Optimierter und physisch ausführbarer Anfrageplan

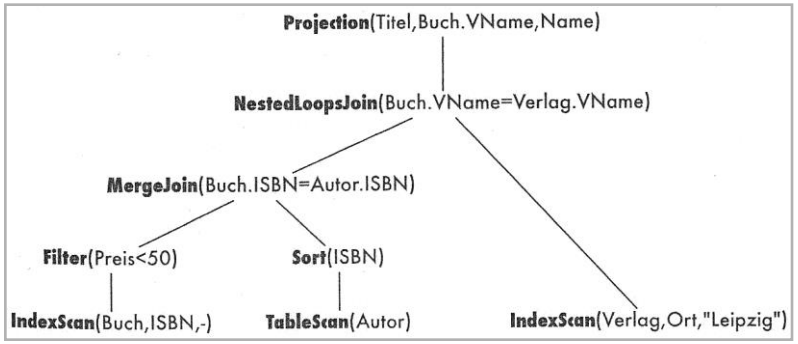
Ausführung des Anfrageplans

Anfrageergebnis

SELECT *Titel*, *VName*, *Name*
FROM *Buch* **NATURAL JOIN** *Autor* **NATURAL JOIN** *Verlag*
WHERE *Preis* < 50 **AND** *Ort* =„Leipzig“







Titel	Vname	Name
...	...	...
...	...	...
...	...	...

(Abbildungen aus: [Ku07], S.251 - 259)

SQL DCL

Ein kleiner Überblick über die SQL DCL (Data Control Language)

- **Zwei wesentliche Aspekte der SQL DCL**
  - **Schutz der Daten in der DB durch Zugriffsberechtigungen**
  - **Transaktionssteuerung für Folgen zusammengehöriger (ändernder) Datenbankoperationen (→ siehe Kapitel 7)**

- **Zugriffsberechtigungen :**
  - **Benutzerverwaltung für das DBMS: Benutzer anlegen und löschen**
    - `CREATE USER` <Benutzername> [`IDENTIFIED BY` [`PASSWORD`] <Passwort>]
    - `DROP USER` <Benutzername> [`CASCADE`]
  - **Vergabe und Entzug von Rechten („Privilegien“)**
    - `GRANT` …
    - `REVOKE` …

SQL DCL
Grundsyntax der Rechtevergabe

```
GRANT    <Recht> {, <Recht>}*
ON       <Datenbankobjekt>  {, <Datenbankobjekt>}*
TO       <Benutzer> {, <Benutzer>}*
WITH     <Rechteoptionen> ;
```

```
REVOKE   <Recht> {, <Recht>}*
ON       <Datenbankobjekt>  {, <Datenbankobjekt>}*
FROM     <Benutzer> {, <Benutzer>}* ;
```

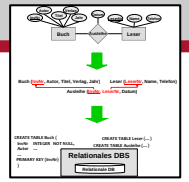
```
<Recht> :=  SELECT [<Spaltenliste>] | INSERT [<Spaltenliste>]     | DELETE
          | UPDATE [<Spaltenliste>] | REFERENCES [<Spaltenliste>] | ALL
```

Hier: Rechte für Tabellen …

```
GRANT SELECT, INSERT
ON    Kunde
TO    MEIER
WITH GRANT OPTION;
```

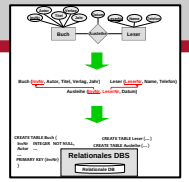
```
GRANT ALL
ON    Kunde
TO    MUELLER;
```

Auf einen Blick: Die verschiedenen Strukturbeschreibungen von Daten und ihr Zusammenhang (zweiter Teil)



Fokus des Datenmodells	Wesentliche Bausteine des Datenmodells	
1 Fachliche Modellierung (ERM, …)	*Entitätstyp (inkl. Schlüssel) + Beziehungstyp*	Konzeptionelles Schema (in Form eines ER-Schemas)
	Überführung / Transformation	
2 Systemnahe Modellierung (Relationales Modell)	*Relationstyp + Schlüssel + Fremdschlüssel*	Logisches Schema (in Form eines relationalen Schemas)
	Überführung / Transformation	
3 Implementierung / System (SQL)	*Tabelle(nstruktur) + Schlüssel + Fremdschlüssel*	Implementierung des Schemas (hier: Tabellenstrukturen)

Auf einen Blick: Die verschiedenen Strukturbeschreibungen mit den zugehörigen Werteebenen (zweiter Teil)



Konzeptionelles Schema

Entitätstyp** (inkl. Schlüssel) + **Beziehungstyp

- **Entitätsmenge pro Entitätstyp (→ Menge von Entitäten)**
- **Beziehungsmenge pro Beziehungstyp: (→ Menge von Beziehungen)**

Logisches Schema

Relationstyp + Schlüssel + Fremdschlüssel

Relation pro Relationstyp (→ Menge von Tupeln)

Internes Schema

Tabelle(nstruktur) + Schlüssel + Fremdschlüssel

Tabelle pro Tabellenstruktur (→ Menge von Zeilen)

(Daten-) Strukturen

Zugehörige (Daten-) Werte

Verständnis- / Lernfragen zu diesem Kapitel

- ***Was sind die Unterschiede zwischen Relation und Tabelle, Attribut und Spalte ?***
- ***Welches sind die wesentlichen SQL-Teilsprachen? Welche zentralen SQL-Anweisungen haben Sie pro Teilsprache kennengelernt? Wie lautet die Grundsyntax und was die Bedeutung der jeweiligen Anweisung?***
- ***Welcher der einzelnen Ebenen der Drei-Schichten-Architektur sind die vorgestellten SQL-Anweisungen zuzuordnen?***
- ***Welches sind die korrespondierenden Begriffe des SQL-Standards für Relation, Tupel, Attribut?***
- ***Was ist der Unterschied zwischen einer prozeduralen und einer deklarativen Sprache? Warum ist dies im Zusammenhang mit SQL von Bedeutung?***
- ***Was ist ein Index und wozu ist er nötig?***
- ***Mit welcher SQL-Anweisung werden Anfragen formuliert? Wie sieht die Grundstruktur einer SQL-Anfrage aus? Was bedeuten die einzelnen Grundbestandteile einer Anfrage?***
- ***Was sind Skalar- bzw. Aggregatfunktionen?***
- ***Wie werden die Verbundoperationen in SQL abgebildet?***
- ***Was sind geschachtelte Anfragen? Welche Prädikate sind hierfür relevant? Wie tief kann man Anfragen schachteln?***
- ***Was hat die Relationenalgebra mit SQL zu tun?***